ADVANCED PROGRAMMING LANGUAGES

Spring 2026, CM 0632. MSc in Computer Science and Information Technology, Ca' Foscari University of Venice

The Simply Typed Lambda Calculus

In our previous lecture, we have introduced a simple static type system for arithmetic expressions. Now we construct a similar type system for the Lambda Calculus.

In the interest of brevity, we ignore most of the operators on integers and booleans and focus on the operators from the language of expressions from our last class.

Syntax and Reduction

We start with the following extended syntax, which includes integers and booleans with their operators, conditionals and a primitive version of the Y combinator. As we will see later on, a primitive recursive operator is needed because the Y combinator from the lambda calculus cannot be typed with the *simple* types we are going to discuss.

The set of terms, noted M, N, P is defined as follows:

$$
\begin{array}{rcll}
M, N, P & ::= & x, y, z & \text{variable} \\
& \mid & \lambda x : T.M & \text{abstraction} \\
& \mid & M\ N & \text{application} \\
& \mid & n, m, k \mid M + N & \text{integers} \\
& \mid & \mathsf{true} \mid \mathsf{false} \mid M = N & \text{booleans} \\
& \mid & \mathsf{if}\ M\ \mathsf{then}\ N\ \mathsf{else}\ P & \text{conditional} \\
& \mid & \mathsf{fix}@T.M & \text{recursion}
\end{array}
$$

We additionally introduce a syntactic category of values:

$$
\begin{array}{rcll}
V & ::= & k, m, n & \text{integers} \\
& \mid & \mathsf{true} \mid \mathsf{false} & \text{booleans} \\
& \mid & \lambda x : T.M & \text{abstraction}
\end{array}
$$

Reduction. We define a set of *top-level reductions* for the primitive operators:

$$
\begin{array}{rcll}
(\lambda x : T.\, M)\, V & \to & [V/x]M & (\beta) \\
n + m & \to & n +_{\mathbb{N}} m & (\text{add}) \\
a = a & \to & \mathsf{true} & (\text{eq-t}) \\
a = b & \to & \mathsf{false} \quad (a \neq b) & (\text{eq-f}) \\
\mathsf{if\ true\ then}\ M\ \mathsf{else}\ N & \to & M & (\text{if-t}) \\
\mathsf{if\ false\ then}\ M\ \mathsf{else}\ N & \to & N & (\text{if-f}) \\
\mathsf{fix}@T.\, M & \to & M\ (\mathsf{fix}@T.\, M) & (\text{fix})
\end{array}
$$

The top level reductions correspond to a call-by-value evaluation strategy, as discussed in our earlier lectures on the lambda calculus and on the language of arithmetic expressions.

The rule for fix implements directly the behavior we established with the fixed-point theorem for the Y combinator: $\mathsf{fix}@T.\, M$ reduces to M applied to itself, i.e. to $M\ (\mathsf{fix}@T.\, M)$, so that $\mathsf{fix}@T.\, M$ is indeed a fixed point of M.

To extend these reductions to arbitrary terms, we define *evaluation contexts* that identify the position within a term where the next reduction step may occur. An evaluation context $\mathcal{E}$ is a term with a designated position. Given a context $\mathcal{E}$ and a term M, we write $\mathcal{E}[M]$ for the term obtained by placing M at that position. I will introduce the notation later

A single *contextual reduction rule* is then all we need to complete the definition of the reduction relation:

$$
\frac{M \to N}{\mathcal{E}[M] \to \mathcal{E}[N]} \quad (\text{ctx})
$$

▶ **Defining Evaluation Contexts**

Types and Typing judgements

To discuss the typing system for the λ-calculus we start by introducing the definition of types:

$$
\begin{array}{rcll}
S, T & ::= & \mathsf{Nat} & \text{natural numbers} \\
& | & \mathsf{Bool} & \text{booleans} \\
& | & S \to T & \text{functions}
\end{array}
$$

As we did for the language of arithmetic expressions, our goal is to define a *typing relation* that associates types to terms.

We will specify this relation by means of an inductive definition for typing judgements of the form $\Gamma \vdash M : T$, to be read "the term M has type T under the assumptions Γ". The *typing environment* (a.k.a *type context*) Γ is a set of type bindings that record the types assumed for the free variables of M.

Typing environments
While we are generally interested in assigning types to closed expressions, for which the simpler judgement form $M : T$ would suffice, the presence of lambda-abstractions requires us to work with type environments as well.

To see why, consider an abstraction $\lambda x.\, M$. To assign a type to it, we need to know what type of arguments to expect for the bound variable x, so that we can determine the type of the body M.

For example, to establish that $\lambda x.\, M : T_1 \to T_2$, we must show that M has type T_2 under the assumption that x has type T_1. Since terms may contain nested abstractions, we need, in general, to maintain several such assumptions simultaneously — this is exactly the role of the type environment Γ in the judgement $\Gamma \vdash M : T$.

Formally, a type environment Γ is a sequence of variables and their types, with " , " as an operator to extend Γ by adding a new binding on the right. The empty context is sometimes written $\emptyset$, but usually we just omit it and write $\vdash M : T$, to state that *the (closed) term M has type T under the empty set of assumptions*.

Type environments may also be thought of as finite maps from variables to their types; following this intuition, we write $\mathrm{dom}(\Gamma)$ for the set of variables bound by Γ.

Type checking vs type inference.
The typing rules we are about to introduce define a relation between contexts, terms, and types.

Given this relation, there are two different algorithmic questions one may ask.

- The first is *type checking*: given a term and a type, verify that the term has that type.
- The second is *type inference*: given a term alone, compute its type.

In the case of type checking, we expect the terms to provide all the information we need. For instance, if we are asked to verify that $\lambda x.\, M : T_1 \to T_2$, the expected type tells us to assume $x : T_1$ and check that $M : T_2$. This is the approach taken by explicitly typed languages, where type annotations — such as $\lambda x : T_1.\, M$ — supply the expected types at each point.

Type inference is harder: if we are simply given $\lambda x.\, M$ and asked to determine its type, we do not know what to assume for x. Implicitly typed languages like Haskell and ML face exactly this problem, and solve it by analyzing how x is used in M to deduce the type.

We will address type checking first, and then move on to type inference later in this lecture.

Before diving into type checking and type inference, we introduce the inductive definition of the typing relation.

Typing Rules

We are ready to introduce the typing rules. In most type systems, each syntactic category of the language is associated with a corresponding form of type judgement. In particular, a common scenario for standard type systems includes at least two forms of judgement, for expressions and for statements. Given that the Lambda Calculus has just expressions, we only have the judgement form we have discussed, for expressions.

Variables. The typing rule for variables follows immediately from the previous discussion on typing environments: a variable has whatever type we are currently assuming it to have.

$$\frac{x : T \in \Gamma}{\Gamma \vdash x : T} \quad (\text{T-Var})$$

Abstraction. The rule for typing abstractions has the following general form:

$$\frac{\Gamma, x : T_1 \vdash M : T_2}{\Gamma \vdash \lambda x : T_1 . M : T_1 \to T_2} \quad (\text{T-Abs})$$

where the premise adds one more assumption in the context to those in the context of the conclusion judgement. To avoid confusion between the new binding for x any bindings that may already appear in Γ, we require that the name x be chosen so that it is distinct from the variables bound by Γ. Since our convention is that variables bound by λ-abstractions may be renamed whenever convenient, this condition can always be satisfied by renaming the bound variable if necessary.

Application. The typing rule for applications is dual to the rule for abstraction:

$$\frac{\Gamma \vdash M : S \to T \qquad \Gamma \vdash N : S}{\Gamma \vdash M\, N : T} \quad (\text{T-App})$$

If M can given a function type $S \to T$, and if N has type S, then the result of applying M to N will be a value of type T.

Recursion. The typing of recursive terms has the following general form:

$$\frac{\Gamma \vdash M : T \to T}{\Gamma \vdash \mathsf{fix}@T.M : T} \quad (\text{T-fix})$$

Notice that M is required to have a function type in which the domain and codomain types coincide: that is consistent with the intended behavior of the fixpoint operator and the type preservation property we expect of the type system, as $\mathsf{fix}.M$ reduces to $M(\mathsf{fix}.M)$.

Constants and operators. The system of inference rules is completed with the direct generalization of the corresponding typing rules for the expression language, adapted to accommodate the context Γ in the judgements so as to account for the presence of variables within the expressions.

$$\frac{}{\Gamma \vdash \mathsf{true} : \mathsf{Bool}} \qquad \frac{}{\Gamma \vdash \mathsf{false} : \mathsf{Bool}} \qquad \frac{}{\Gamma \vdash n : \mathsf{Nat}}$$

$$\frac{\Gamma \vdash M : \mathsf{Nat} \quad \Gamma \vdash N : \mathsf{Nat}}{\Gamma \vdash M + N : \mathsf{Nat}} \qquad \frac{\Gamma \vdash M : T \quad \Gamma \vdash N : T}{\Gamma \vdash M = N : \mathsf{Bool}}$$

$$\frac{\Gamma \vdash M : \mathsf{Bool} \quad \Gamma \vdash N : T \quad \Gamma \vdash P : T}{\Gamma \vdash \mathsf{if}\ M\ \mathsf{then}\ N\ \mathsf{else}\ P : T}$$

Type Safety

As we have established in our previous lecture, the fundamental property of type systems is *safety* (or soundness), summarized with the slogan *well-typed terms do not go wrong*. The formalization consists of two theorems: Progress and Preservation.

Theorem [Progress]
Suppose M is a closed, well-typed term, that is $\vdash M : T$ for some T. Then either M is a value or else there exists N such that $M \to N$.

Theorem [Preservation] If $\Gamma \vdash M : T$ and $M \to N$ then $\Gamma \vdash N : T$.

The Progress theorem tells us that well-typed terms are not stuck: they are either values or they can take a step. The Preservation theorem guarantees that reduction preserves types: if a well-typed term reduces, the result is still well-typed, with the same type.

Together, the two theorems ensure that evaluation of a well-typed term can never reach a stuck state.

The proofs follow the same structure as for the language of arithmetic expressions: they proceed by rule induction on the typing derivation. The main difference is that we now need to account for variables, abstractions and applications, which requires us to establish a few structural properties of typing contexts. In particular, the proof of preservation relies on a new result — the substitution lemma — which states that the typing relation is preserved when variables are replaced by terms of appropriate types:

Lemma [Preservation of Types Under Substitution]
If $\Gamma, x : S \vdash M : T$ and $\Gamma \vdash N : S$, then $\Gamma \vdash [N/x]M : T$.

This lemma is needed for the β-reduction case of the preservation proof: when a well-typed application $(\lambda x : S.M)\ N$ reduces to $[N/x]M$, we need to know that the substitution preserves the type of the body. We omit the proofs and refer the interested reader to Chapter 9 of Pierce's *Types and Programming Languages*.

From rules to algorithms: type checking

So far, we have presented the typing rules as a *declarative specification*: they define a relation between contexts, terms, and types, and they tell us *what* it means for a term to be well-typed. But they don't tell us *how* to decide whether a given term is well-typed.

In this section and the next, we address the algorithmic question. We will see that the same set of rules can be read in two different ways, corresponding to two different tasks:

- *type checking* (given a term with type annotations, verify that it is well-typed)
- *type inference* (given a term without annotations, reconstruct its type).

This distinction corresponds precisely to the difference between explicitly typed and implicitly typed languages.

Syntax-directedness

The first key observation is that our typing rules are *syntax-directed*: for each syntactic form of the language, there is exactly one typing rule whose conclusion matches that form. Variables are handled by (T-Var), abstractions by (T-Abs), applications by (T-App), and so on. There is no overlap and no ambiguity: given a term, its syntactic form uniquely determines which rule must be applied.

This means that if we read the rules *bottom-up* — from conclusion to premises — they become a *deterministic algorithm*. Given a context Γ and a term M, we look at the shape of M, select the

corresponding rule, and recursively check the premises. The algorithm either succeeds (and returns the type of M) or fails (reporting a type error).

The type checking algorithm

Let us define a function $\mathsf{typeof}(\Gamma, M)$ that, given a context and a term, either returns the type of M or signals an error. The definition follows the typing rules, one case per syntactic form:

- $\mathsf{typeof}(\Gamma, x)$:
  look up x in Γ.
  If $x : T \in \Gamma$, return T; otherwise, error (unbound variable).

- $\mathsf{typeof}(\Gamma, \lambda x : T_1 . M)$:
  let $T_2 = \mathsf{typeof}(\Gamma \cup \{x : T_1\},\ M)$.
  return $T_1 \to T_2$.

- $\mathsf{typeof}(\Gamma,\ M\ N)$:
  let $T_M = \mathsf{typeof}(\Gamma, M)$
  let $T_N = \mathsf{typeof}(\Gamma, N)$.
  check that T_M has the form $S \to T$ and that $T_N = S$.
  if so, return T; otherwise, error.

- $\mathsf{typeof}(\Gamma,\ n)$:
  return Nat.

- $\mathsf{typeof}(\Gamma,\ \mathsf{true})$:
  return Bool.

- $\mathsf{typeof}(\Gamma,\ \mathsf{false})$:
  return Bool.

- $\mathsf{typeof}(\Gamma,\ M + N)$:
  let $T_M = \mathsf{typeof}(\Gamma, M)$
  let $T_N = \mathsf{typeof}(\Gamma, N)$.
  check that $T_M = T_N = \mathsf{Nat}$.
  if so, return Nat; otherwise, error.

- $\mathsf{typeof}(\Gamma,\ M = N)$:
  let $T_M = \mathsf{typeof}(\Gamma, M)$

let $T_N = \mathsf{typeof}(\Gamma, N)$.
if $T_M = T_N$ return Bool else return error (type mismatch)

- $\mathsf{typeof}(\Gamma,\ \mathsf{if}\ M\ \mathsf{then}\ N\ \mathsf{else}\ P)$:
  let $T_M = \mathsf{typeof}(\Gamma, M)$
  let $T_N = \mathsf{typeof}(\Gamma, N)$
  let $T_P = \mathsf{typeof}(\Gamma, P)$.
  check that $T_M = \mathsf{Bool}$ and $T_N = T_P$.
  if so, return T_N; otherwise, error (type mismatch)

- $\mathsf{typeof}(\Gamma,\ \mathsf{fix}@T.M)$:
  let $T_M = \mathsf{typeof}(\Gamma, M)$.
  check that $T_M = T \to T$.
  if so, return T; otherwise, error.

The algorithm is a simple recursive traversal of the term.

If you compare each case with the corresponding typing rule, you will see that the algorithm is just the rule read bottom-up: the conclusion tells us *which* case we are in, and the premises tell us *what to check recursively*.

This is essentially what a type checker does in an explicitly typed language such as Scala: the programmer provides all the type annotations, and the compiler walks through the program, verifying that the annotations are consistent. In Scala, for example:

```
val double: Int => Int = (x: Int) => x + x
```

corresponds to checking that $\mathsf{typeof}(\emptyset,\ \lambda x : \mathsf{Nat}.\ x + x) = \mathsf{Nat} \to \mathsf{Nat}$

The annotation Nat on x tells the type checker what to assume, and the algorithm verifies that the body $x + x$ indeed has type Nat under this assumption.

From type checking to type inference

Now consider an implicitly typed language, where the programmer does not annotate the bound variables. In Haskell, for example, one writes:

```
double = \x -> x + x
```

without specifying the type of `x`. The corresponding lambda term is $\lambda x.\, x + x$ — with no annotation on x. The Haskell compiler must figure out on its own that `x` must be of numeric type.

The problem is clear if we try to apply our type checking algorithm to $\lambda x.\, x + x$. At the abstraction case, we need to extend the context with a binding $x : T_1$ — but the term does not tell us what T_1 is. The algorithm, as defined, gets stuck.

The idea: type variables as placeholders

The solution is to introduce *type variables* — placeholders for the types we do not yet know — and let the rules themselves *generate constraints* that these variables must satisfy.

We extend the syntax of types with type variables $\alpha, \beta, \gamma, \ldots$ which stand for unknown types:

$$\begin{array}{rcll} S, T & ::= & \mathsf{Nat} & \text{natural numbers} \\ & | & \mathsf{Bool} & \text{booleans} \\ & | & S \to T & \text{functions} \\ & | & \alpha, \beta, \gamma, \ldots & \text{type variables} \end{array}$$

and we modify the syntax of terms to drop the annotation from the abstraction and from fix:

$$\begin{array}{rcll} M, N, P & ::= & \ldots & \\ & | & \lambda x.M & \text{abstraction (no annotation)} \\ & | & \mathsf{fix}.M & \text{recursion (no annotation)} \end{array}$$

Now, instead of checking types, we *infer* them.

The algorithm traverses the term as before, but

- Whebn it needs a type that is not provided, it introduces a fresh type variable.
- When the rules require two types to match, instead of checking equality, the algorithm records an *equation* (a constraint) between the two types.

At the end of the traversal, we are left with a type expressed in terms of type variables, and a set of constraints.

The question then becomes: is there an assignment of concrete types to the type variables that satisfies all the constraints simultaneously?

The inference algorithm

We redefine our algorithm as a function $\mathsf{infer}(\Gamma,\ M)$ that returns a pair $(T, \mathcal{C})$ where T is a type (possibly containing type variables) and $\mathcal{C}$ is a set of equations between types.

- $\mathsf{infer}(\Gamma,\ x)$:
  look up x in Γ.
  if $x : T \in \Gamma$, return $(T,\ \emptyset)$ else error.

- $\mathsf{infer}(\Gamma,\ \lambda x.\ M)$:
  introduce a fresh type variable α.
  let $(T_M,\ \mathcal{C}) = \mathsf{infer}(\Gamma \cup \{x : \alpha\},\ M)$.
  return $(\alpha \to T_M,\ \mathcal{C})$.

- $\mathsf{infer}(\Gamma,\ M\ N)$:
  introduce a fresh type variable α.
  let $(T_M,\ \mathcal{C}_M) = \mathsf{infer}(\Gamma,\ M)$
  let $(T_N,\ \mathcal{C}_N) = \mathsf{infer}(\Gamma,\ N)$.
  return $(\alpha,\ \mathcal{C}_M \cup \mathcal{C}_N \cup \{T_M = T_N \to \alpha\})$.

- $\mathsf{infer}(\Gamma,\ n)$:
  return $(\mathsf{Nat},\ \emptyset)$.

- $\mathsf{infer}(\Gamma,\ \mathsf{true})$
  return $(\mathsf{Bool},\ \emptyset)$.

- $\mathsf{infer}(\Gamma,\ \mathsf{false})$:
  return $(\mathsf{Bool},\ \emptyset)$.

- $\mathsf{infer}(\Gamma,\ M + N)$:
  let $(T_M,\ \mathcal{C}_M) = \mathsf{infer}(\Gamma,\ M)$
  let $(T_N,\ \mathcal{C}_N) = \mathsf{infer}(\Gamma,\ N)$.
  return $(\mathsf{Nat},\ \mathcal{C}_M \cup \mathcal{C}_N \cup \{T_M = \mathsf{Nat},\ T_N = \mathsf{Nat}\})$.

- $\mathsf{infer}(\Gamma,\ M = N)$:
  let $(T_M,\ \mathcal{C}_M) = \mathsf{infer}(\Gamma,\ M)$
  let $(T_N,\ \mathcal{C}_N) = \mathsf{infer}(\Gamma,\ N)$.

return $(\mathsf{Bool},\ \mathcal{C}_M \cup \mathcal{C}_N \cup \{T_M = T_N\})$.

- $\mathsf{infer}(\Gamma,\ \mathsf{if}\ M\ \mathsf{then}\ N\ \mathsf{else}\ P)$:
  let $(T_M,\ \mathcal{C}_M) = \mathsf{infer}(\Gamma,\ M)$
  let $(T_N,\ \mathcal{C}_N) = \mathsf{infer}(\Gamma,\ N)$
  let $(T_P,\ \mathcal{C}_P) = \mathsf{infer}(\Gamma,\ P)$.
  return $(T_N,\ \mathcal{C}_M \cup \mathcal{C}_N \cup \mathcal{C}_P \cup \{T_M = \mathsf{Bool},\ T_N = T_P\})$.

- $\mathsf{infer}(\Gamma,\ \mathsf{fix}.\ M)$:
  introduce a fresh type variable α.
  let $(T_M,\ \mathcal{C}) = \mathsf{infer}(\Gamma,\ M)$.
  return $(\alpha,\ \mathcal{C} \cup \{T_M = \alpha \to \alpha\})$.

A worked example

Let us trace the algorithm on the term $\lambda x.\ x + 1$.

Let us trace the algorithm on the term $\lambda x.\ x + 1$, starting with

$\mathsf{infer}(\emptyset,\ \lambda x.\ x + 1)$: the term is an abstraction, so introduce a fresh variable α and compute

$\mathsf{infer}(\{x : \alpha\},\ x + 1)$: the term is an addition, so compute

$\mathsf{infer}(\{x : \alpha\},\ x) = (\alpha,\ \emptyset)$

$\mathsf{infer}(\{x : \alpha\},\ 1) = (\mathsf{Nat},\ \emptyset)$.

return $(\mathsf{Nat},\ \{\alpha = \mathsf{Nat},\ \mathsf{Nat} = \mathsf{Nat}\})$.

Back in the abstraction, return $(\alpha \to \mathsf{Nat},\ \{\alpha = \mathsf{Nat},\ \mathsf{Nat} = \mathsf{Nat}\})$.

The constraint $\alpha = \mathsf{Nat}$ (the other equation is trivially true) tells us that α must be Nat. Applying this substitution to the inferred type $\alpha \to \mathsf{Nat}$, we obtain $\mathsf{Nat} \to \mathsf{Nat}$.

A slightly more interesting example

Consider the term $\lambda f.\ \lambda x.\ f\ (f\ x)$, the "apply twice" function. We trace the algorithm starting with

$\mathsf{infer}(\oslash,\ \lambda f.\ \lambda x.\ f\ (f\ x))$: the term is an abstraction, so introduce a fresh variable α for f and compute

$\mathsf{infer}(\{f : \alpha\},\ \lambda x.\ f\ (f\ x))$: again an abstraction, so introduce a fresh variable β for x and compute

$\mathsf{infer}(\{f : \alpha,\ x : \beta\},\ f\ (f\ x))$: the term is an application, so introduce a fresh variable γ for the result and compute

$\mathsf{infer}(\{f : \alpha,\ x : \beta\},\ f) = (\alpha,\ \emptyset)$

$\mathsf{infer}(\{f : \alpha,\ x : \beta\},\ f\ x)$: another application, so introduce a fresh variable δ for the result and compute:

$\mathsf{infer}(\{f : \alpha,\ x : \beta\},\ f) = (\alpha,\ \emptyset)$

$\mathsf{infer}(\{f : \alpha,\ x : \beta\},\ x) = (\beta,\ \emptyset)$

return $(\delta,\ \{\alpha = \beta \to \delta\})$.

Back in the outer application, return $(\gamma,\ \{\alpha = \beta \to \delta,\ \alpha = \delta \to \gamma\})$.

Back in the inner abstraction, return $(\beta \to \gamma,\ \{\alpha = \beta \to \delta,\ \alpha = \delta \to \gamma\})$.

Back in the outer abstraction, return $(\alpha \to \beta \to \gamma,\ \{\alpha = \beta \to \delta,\ \alpha = \delta \to \gamma\})$.

Now we solve the constraints. From the two equations we have $\beta \to \delta = \delta \to \gamma$, which gives us $\beta = \delta$ and $\delta = \gamma$. Writing β for the common value, we get $\alpha = \beta \to \beta$.

Applying this substitution to the inferred type $\alpha \to \beta \to \gamma$, we obtain $(\beta \to \beta) \to \beta \to \beta$.

This is indeed the most general type of the "apply twice" function: it takes a function from β to β and a value of type β, and returns a value of type β.

What remains: solving the constraints

In the examples above, the constraints were simple enough that we could solve them by hand. In general, however, the constraint set can be large and the equations can involve nested type expressions. What we need is an algorithm that takes a set of type equations and finds, if one exists, the *most general* substitution of types for type variables that satisfies all the constraints.

This problem is known as *unification*, and its solution — due to Robinson (1965) — is the algorithm at the heart of type inference in languages like Haskell, ML, and OCaml.

The key result we are aiming for is the following: if the constraint set has a solution, then it has a *most general* one — a substitution from which all other solutions can be obtained by further substitution.

This most general solution is called the *most general unifier*, and the type it produces (by applying the unifier to the inferred type) is the *principal type* of the term. In our first example, $\mathsf{Nat} \to \mathsf{Nat}$ was the unique type; in our second example, $(\beta \to \beta) \to \beta \to \beta$ is the principal type — the most general type from which any valid typing of "apply twice" can be obtained by substituting a concrete type for β.

Summary

We have presented the typing rules of the simply typed lambda calculus in three different modes:

- as a *declarative specification*: a set of inference rules that define the typing relation $\Gamma \vdash M : T$;
- as a *type checking algorithm*: reading the rules bottom-up on explicitly typed terms yields a deterministic recursive procedure — this is what languages like Scala and Java do;
- as a *type inference algorithm*: on implicitly typed terms, the same rules generate constraints between type variables, to be solved by unification — this is what languages like Haskell and ML do.

In our next lecture we will study unification and complete the type inference algorithm.

Exercises

1. Subject expansion? Having seen the subject reduction property, it is reasonable to wonder whether the opposite property — subject expansion — also holds. Is it always the case that, if $M \to N$ and $\Gamma \vdash N : T$ then $\Gamma \vdash M : T$? If so, prove it. If not, give a counterexample.

2. Typing of conditionals. As we argued in the previous lecture, the ill-typed terms of our expression language include terms like
$\mathsf{if\ true\ then\ 0\ else\ false}$ that actually behave fine during evaluation. Show that if we change the typing of conditionals to make the previous term well-typed we can construct a well-typed term that gets stuck.

3. Typing of equality. In our typing rule for the equality term $M = N$ we require that M and N have the same type.

- How would you change the rule to relax that constraint (and admit the equality test on terms of different types)?
- Which of the properties of the original system would change with the new rule?
- In particular, would the new system still be sound? If so, prove it. Otherwise provide a counterexample.

4. Type checking by hand. Apply the typeof algorithm to the following terms and determine their types (or report a type error):

- $\lambda x : \mathsf{Nat}.\ \mathsf{if}\ (x = 0)\ \mathsf{then}\ \mathsf{true}\ \mathsf{else}\ \mathsf{false}$
- $\lambda f : (\mathsf{Nat} \to \mathsf{Bool}).\ \lambda x : \mathsf{Nat}.\ f\ (x + 1)$
- $(\lambda x : \mathsf{Nat}.\ x)\ \mathsf{true}$

5. Type inference by hand. Apply the infer algorithm to the following terms and write down the inferred type and the constraint set. Then solve the constraints by hand to find the principal type:

- $\lambda x.\ x$
- $\lambda f.\ \lambda x.\ f\ x$
- $\lambda f.\ \lambda x.\ f\ x + 1$

6. Untypability of Ω. Recall that the term $\omega = \lambda x.\ x\ x$ plays a central role in the construction of the Ω combinator ($\Omega = \omega\ \omega$) and of the fixed-point combinator Y.

Apply the infer algorithm to the term $\lambda x.\ x\ x$. Write down the inferred type and the constraint set. Show that the constraints have no solution, and explain why.

Hint: look at the shape of the constraint — what would it require of a type to be equal to one of its own sub-expressions?

Use this result to argue informally why the Y combinator $\lambda f.\ (\lambda x.\ f\ (x\ x))\ (\lambda x.\ f\ (x\ x))$ cannot be typed either, and why the simply typed lambda calculus therefore needs a primitive fix operator for recursion.